

Position Generalization in Translated FashionMNIST

Architecture, patch scale, and patch embedding

Final English report

12 fitted models / 24 test evaluations

Abstract. We test whether image classifiers retain accuracy when object position changes between training and testing. Across six configurations and one shared training recipe, CNN records the highest accuracy and reaches 35.40% on the hardest fixed-to-random transfer. Patch size 8 leads three of four ViT settings. Equivalent Conv2d and Flatten + Linear projections remain within 0.61 percentage points in this run.

1 Question

A classifier may learn class appearance and object location together. We isolate location by placing the same 28 x 28 object either uniformly over all valid integer positions (A) or at the canvas center (B).

Setting	Training	Testing	Purpose
A -> A	random	random	in-distribution reference
B -> B	center	center	fixed-position reference
A -> B	random	center	random-to-fixed transfer
B -> A	center	random	hardest position shift

2 Controlled protocol

Each of six configurations is fitted once on A and once on B. Validation selects the checkpoint; both A and B test sets are then measured, producing 12 fits and 24 final test evaluations.

Item	Controlled choice
Data	28 x 28 FashionMNIST image on a 64 x 64 black canvas
Split	54,000 train, 6,000 validation; official 10,000-image test set held out
Training	15 epochs; batch 64; AdamW, lr 1e-3, weight decay 1e-4
Selection	best validation checkpoint
Reporting	top-1 test accuracy; elapsed training time; seed 42

Primary measure. B -> A is the hardest setting because a model trained only on centered objects must classify objects across the canvas.

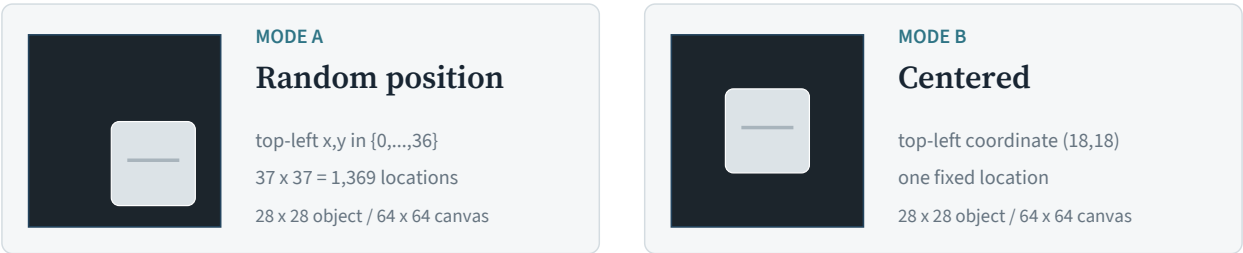


Figure 1. Mode A placements are deterministic per sample and seed; the same test placement seed is used for every configuration.

Architecture and Position Shift

MLP, CNN, and patch-8 ViT under one evaluation protocol

3 Accuracy

Configuration	Parameters	A -> A	B -> B	A -> B	B -> A
MLP	542,474	71.89	89.57	71.97	13.99
CNN	205,994	92.35	93.14	92.87	35.40
ViT, patch 8	811,402	80.40	88.27	81.13	18.81

Table 1. Top-1 test accuracy (%); teal values mark column leaders.

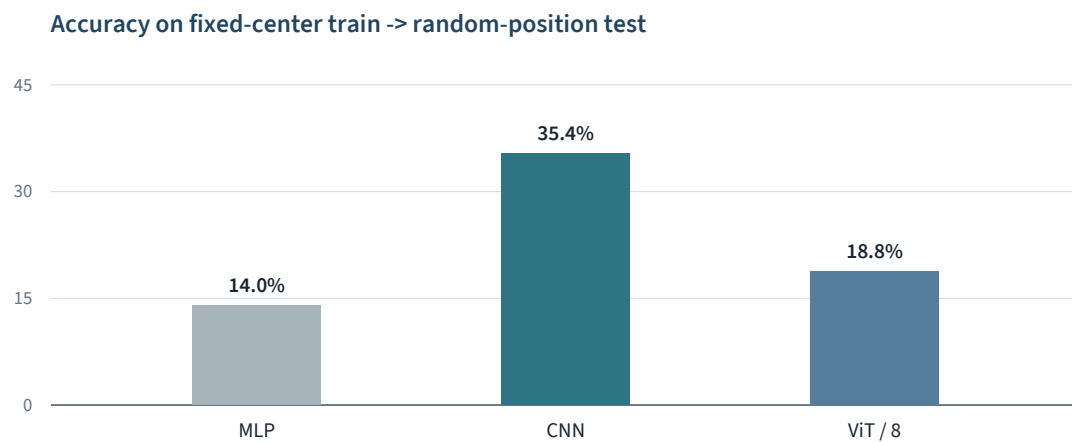


Figure 2. CNN retains substantially more accuracy on B -> A.

BEST B -> A 35.40% CNN	CNN ADVANTAGE +16.59 pp over patch-8 ViT	SMALLEST MODEL 205,994 CNN parameters
---	--	---

4 Effect of the position shift

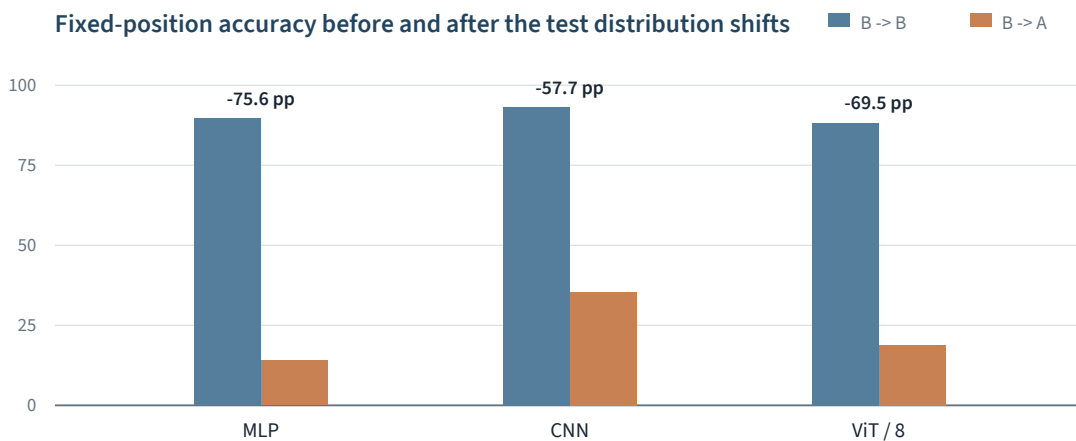


Figure 3. All models degrade after centered training is tested at random positions.

Patch Scale and Embedding

Two controlled changes to the shared Transformer configuration

5 Patch scale

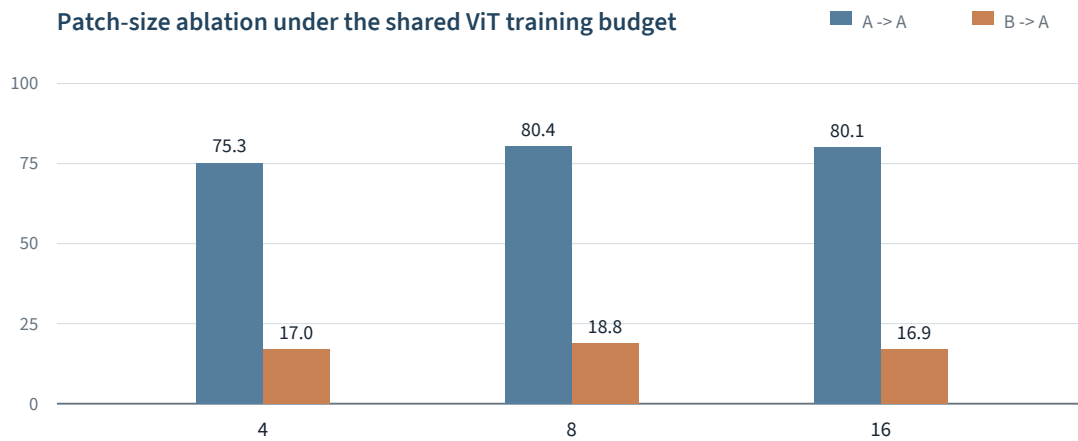


Figure 4. Patch size 8 leads three of four evaluated settings.

Patch	Tokens	A -> A	B -> B	A -> B	B -> A	Time
4	256	75.28	87.22	74.97	17.02	8.93 min
8	64	80.40	88.27	81.13	18.81	7.36 min
16	16	80.07	88.87	80.97	16.91	6.46 min

Table 2. Accuracy (%) and summed training time; teal marks leaders and fastest time.

Result. Patch size 8 leads on A -> A, A -> B, and B -> A. Patch size 4 raises the token count to 256 and is slower without an accuracy gain.

6 Patch embedding

For non-overlapping patches, Conv2d with kernel = stride = p and a shared Linear layer on each flattened $p \times p$ patch implement the same affine map when their weights are aligned. Their trained results remain close.

Test	Conv2d	Flatten + Linear	Absolute gap
A -> A	80.07	79.67	0.40
B -> B	88.87	88.80	0.07
A -> B	80.97	80.48	0.49
B -> A	16.91	16.30	0.61

Table 3. Test accuracy (%) for patch size 16. Maximum absolute gap: 0.61 points.

Conclusions and Reproducibility

Concise interpretation and a documented reproduction path

7 **Conclusions**

01	CNN performs best under the shared recipe. It leads all four settings and retains the most fixed-to-random accuracy.
02	Patch size 8 leads three of four settings. It gives the best B -> A result without the 256-token cost of patch size 4.
03	Equivalent embeddings stay close in this run. Conv2d and Flatten + Linear differ by at most 0.61 percentage points.

8 **Limitations**

Scope	Interpretation rule
- One training and placement seed; no variance estimate. - Model sizes and optimal hyperparameters are not matched. - Runtime is hardware- and order-specific.	Small differences are described only as numerically close in this run; no significance claim is made. Conclusions apply to the shared protocol rather than all possible training recipes.

9 **Reproducibility**

Check	Recorded value
Code	translated_fashionmnist/experiments
Metrics	results/metrics.csv
Design	12 fits, 24 test evaluations; seed 42; 15 epochs
Dependency lock	requirements-lock.txt
Environment	Python 3.12.13, PyTorch 2.11.0, CUDA 12.8
Hardware	NVIDIA GeForce RTX 5070 Laptop GPU

```
python -m translated_fashionmnist.experiments.compare --groups all --download
```

10 **Implementation map**

Component	Location	Role
Dataset	data.py	load, split, and construct A/B canvases
Models	models.py	MLP, CNN, ViT, patch embeddings
Protocol	experiments/protocol.py	train, validate, and evaluate
Engine	engine.py	shared training and evaluation primitives
Runner	experiments/compare.py	execute all three comparison groups
Outputs	results/	metrics, history, and figures